

Application du traitement de signal sur graphe à la classification

September 19, 2015

Khalid AITELAMINE
Encadrant: Bastien PASDELOUP

Contents

1	Introduction	2
2	Objectif	2
3	Les données	3
3.1	Représentation des données	3
3.2	Traitement des données	3
3.2.1	Subdivision	3
3.2.2	Parcours par une fenêtre glissante	3
3.2.3	Extraction d'une partie de l'image	3
3.2.4	Extraction des descripteurs à partir d'une image complète	4
4	Outils des théories des graphes	4
4.1	Graphe	4
4.1.1	Définition	4
4.1.2	Laplacian d'un graphe	4
4.1.3	Signal sur graphe	4
5	Algorithme d'apprentissage du Laplacian	5
5.1	Objectif	5
5.2	Le problème d'optimisation	5
6	Application à MNIST	7
6.1	Description	7
6.2	Résultats	7
7	Conclusion	8

1 Introduction

Avec les moyens technologiques disponibles aujourd’hui, nous pouvons collecter une grande quantité de données rapidement, ses données sont souvent considérées comme dynamiques et elles ne peuvent pas être représentées dans des espaces mathématiques tel qu’un espace euclidien pour pouvoir y appliquer les outils de traitement de signal classiques, nous pouvons citer comme exemple de données la température dans différentes régions, les partages et les messages des utilisateurs des réseaux sociaux ...

D’autre part, ces données sont en général dépendantes, autrement dit, il faut tenir en compte les interrelations existantes entre eux, par suite, il s’avère très judicieux de représenter ces données sous forme d’un graphe et généraliser les méthodes et les techniques classiques de traitement du signal à ce type de données, d’où l’apparition d’un nouveau domaine appelé “Traitement de signal sur graphe”, ce domaine est très récent, il a inspiré beaucoup de chercheurs et scientifiques et nous pouvons en trouver beaucoup d’articles scientifiques, nous citons par exemple l’article [1].

Quant à la classification, c’est une méthode qui a pour objectif le regroupement d’un ensemble d’individus ou d’objets, qui sont représentés sous forme de données, ces données peuvent être de n’importe quel type, par exemple une image, des textes, des gènes ...

En langage mathématiques, la classification est l’opération de partitionner un ensemble de données en plusieurs classes (clusters) disjointes et homogènes, en d’autres termes, les éléments d’une même classe ont les mêmes propriétés selon des critères de similarités données, ces critères sont traduits en des mesures des distances entre données, sachant que les données sont représentés sous forme de vecteurs ayant une dimension d .

Bien évidemment, on peut trouver un moyen qui permet d’employer les mécanismes du traitement de signal sur graphe pour la classification, à titre d’exemple l’algorithme de “classification spectrale” ou en anglais “spectrale clustering”.

2 Objectif

L’objectif de ce stage est d’adopter les outils de traitement de signal sur graphe pour la classification et l’apprentissage, plus précisément, étant donné un ensemble de données avec les labels correspondant, il s’agit de construire un graphe pour chaque label et ensuite, utiliser ces graphes pour prédire la valeur du label de chaque élément. Ce stage concerne en générale la base de données MNIST, qui consiste en un ensemble d’images de chiffres manuscrits, le but est de construire le graphe associé à chaque chiffre, ensuite prédire la valeur du chiffre de chaque image.

3 Les données

3.1 Représentation des données

Les données sont en général des images, une image est une matrice M de taille $q \times q$, l'ensemble des données est un couple de matrices $(X, Lab) \in M_{n \times p}(\mathbb{R}) \times M_n(\mathbb{R})$, chaque ligne de cette matrice est un descripteur d'une image de la base de donnée, donc les descripteurs sont de dimensions p , un descripteur sera considéré désormais comme un signal. La matrice Lab est un vecteur colonne qui contient les labels de chaque ligne de la matrice X .

Elles sont également séparées en deux parties, la première est un ensemble de descripteurs pour le trainement, la deuxième et pour les tests.

3.2 Traitement des données

Vu que la dimension des descripteurs pourra être grande, nous avons donc pensé un premier lieu à gérer ces données, voire à décomposer chaque descripteur afin de réduire la complexité de notre algorithme, nous allons représenter ici quelques idées que nous avons utilisé.

3.2.1 Subdivision

Soit x un descripteur, x est normalement un vecteur colonne de taille p , il représente une image de notre base de données, à partir de x on peut donc obtenir la matrice M de notre image, nous allons ensuite diviser cette matrice en $m \times m$ blocs, i.e $M = (M_{ij})_{1 \leq i, j \leq m}$.

3.2.2 Parcours par une fenêtre glissante

Dans cette partie, nous avons appliqué l'opération suivante aux descripteurs:

Soit x un descripteur de notre base de données, $M \in M_{q \times q}(\mathbb{R})$ sa matrice image correspondante, il s'agit de parcourir la matrice M par une fenêtre (matrice) de taille $h \times w$, avec des pas de translations horizontal et vertical t_h et t_w respectivement, donc au total nous aurons $(\frac{q-h}{t_h} + 1)(\frac{q-w}{t_w} + 1)$ sous-matrices extraites de l'image M .

3.2.3 Extraction d'une partie de l'image

Soit x un descripteur de notre base de données, $M \in M_{q \times q}(\mathbb{R})$ sa matrice image correspondante, nous pouvons utiliser seulement une partie de cette matrice, autrement dit, on peut considérer la matrice $\widetilde{M} = M(\frac{q-\tilde{q}}{2} + 1 : \tilde{q} + \frac{q-\tilde{q}}{2}, \frac{q-\tilde{q}}{2} + 1 : \frac{q-\tilde{q}}{2} + \tilde{q})$ avec $\tilde{q} < q$. cette méthode sera utilisé dans le code vu qu'elle donne des résultats plus bons.

3.2.4 Extraction des descripteurs à partir d'une image complète

La méthode qui sera utilisée ici est appelée histogramme du gradient orienté pour extraire de chaque image un descripteur, et puis on applique l'algorithme à la nouvelle matrice obtenue.

Le principe de la fonction HOG (Histogram of Oriented Gradient) est de diviser l'image en des régions ou des cellules, puis calculer les histogrammes des directions du gradient pour chaque cellule, et tous ces histogrammes forment le descripteur voulu [2].

4 Outils des théories des graphes

4.1 Graphe

4.1.1 Définition

Un graphe est un triplé $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathcal{W}\}$, avec $\mathcal{V}$ est un ensemble fini de sommets, $\mathcal{E}$ est l'ensemble des arêtes entre les sommets et $\mathcal{W}$ est la matrice d'adjacence du graphe qui contient les poids des arêtes.

Dans toute la suite, nous ne considérons que les graphes non orientés, c'est-à-dire que $\mathcal{W}$ est symétrique et que si $(i, j) \in \mathcal{E}$ alors $(j, i) \in \mathcal{E}$.

Si nous disposerions seulement du couple $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, alors on peut définir $\mathcal{W}$ par: $\mathcal{W} = \begin{cases} \exp(-\frac{d(i,j)^2}{2\theta^2}), & \text{si } d(i,j) \leq \kappa \text{ et} \\ 0 & \text{sinon.} \end{cases}$

4.1.2 Laplacien d'un graphe

Soit le graphe $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathcal{W}\}$, on définit la matrice des degrés par $D = (d_{ij})_{1 \leq i, j \leq n}$ avec $n = |\mathcal{V}|$ et $d_{ij} = \delta_{ij} \sum_{k=1}^n w_{ik}$ avec δ le symbole de Kronecker.

Le Laplacien du graphe $\mathcal{G}$ est défini par: $L = D - W$. L est une matrice réelle symétrique, donc diagonalisable dans une base orthonormale: $L = \chi \Lambda \chi^T$.

Λ est une matrice diagonale des valeurs propres de L et χ est la matrice formée par la base orthonormale des vecteurs propres de L .

La formule suivante sera utilisée dans la suite: soit $f \in \mathbb{R}^n$, alors $S_2(f) = f^T L f = \frac{1}{2} \sum_{i,j} w_{ij} (f(i) - f(j))^2$. Ce paramètre est appelé la forme 2-Dirichlet discrète.

4.1.3 Signal sur graphe

Un signal sur un graphe $\mathcal{G} = \{\mathcal{V}, \mathcal{E}, \mathcal{W}\}$ est une fonction $f : \mathcal{V} \rightarrow \mathbb{R}, i \mapsto f(i)$.

un signal f est dit lisse sur le graphe $\mathcal{G}$ si $S_2(f) = f^T L f$ est minimal, ou encore si l'énergie f est concentrée sur les premières petites valeurs propres de L .

- D'après une littérature [4], nous allons adopter la représentation suivante pour les signaux que nous allons gérer: si x est un signal, alors il sera écrit sous la forme $x = \chi h + u_x + \epsilon$ avec h est une variable de latence

qui contrôle le signal x via la base des vecteurs propres du Laplacien L du graphe, u_x est la moyenne de x et ϵ est un bruit Gaussien centré de matrice de covariance $\sigma_\epsilon^2 I_n$, i.e $\epsilon \sim \mathcal{N}(0, \sigma_\epsilon^2 I_n)$. Est condition supplémentaire est ajouté est que la variable $h \sim \mathcal{N}(0, \Lambda^\dagger)$ avec $\Lambda^\dagger$ est la matrice pseudo-inverse de la matrice des valeurs propres Λ de L , cette condition sur h signifie que l'énergie du signal est concentré sur les basses fréquences ou bien sur les premières petites valeurs propres, pour que le signal x soit lisse sur le graphe $\mathcal{G}$.

5 Algorithmes d'apprentissage du Laplacien

5.1 Objectif

Le but de cette partie est de présenter un algorithme d'une littérature qui permet de déterminer la topologie du graphe sur lequel un ensemble de signaux $X \in M_{n \times p}(\mathbb{R})$ seront lisse, la matrice X contient n signaux de dimension p , donc nous allons chercher à déterminer une matrice Laplacien L d'un graphe tel que les signaux de X soient lisse sur ce graphe, pour celà, il faut résoudre le problème d'optimisation suivant : [4].

5.2 Le problème d'optimisation

Il s'agit de résoudre le problème suivant:

$$\min_{L \in M_{p \times p}(\mathbb{R}), Y \in M_{n \times p}(\mathbb{R})} \|X - Y\|_F^2 + \alpha \text{Tr}(YLY^T) + \beta \|L\|_F^2$$

$$t.q \quad \text{Tr}(L) = p, L_{ij} = L_{ji} \leq 0 \ i \neq j, L\mathbf{1} = \mathbf{0}$$

Y est une nouvelle représentation des signaux de X , α et β sont deux paramètres de régularisations qui sont positifs. La contrainte sur la trace de L permet d'éviter les solutions nulles pendant la programmation, on peut mettre $\text{Tr}(L) = m$ avec $m > 0$.

On peut remarquer que ce problème n'est pas convexe, mais il en est pour chacune des variables (càd on fixe une variable et on optimise par rapport à l'autre). C'est pour cela qu'on va diviser ce problème en deux:

Le premier sert à déterminer L en fixant Y . donc

$$\min_{L \in M_{p \times p}(\mathbb{R})} \alpha \text{Tr}(YLY^T) + \beta \|L\|_F^2 \quad (1)$$

$$s.t \quad \text{Tr}(L) = p, L_{ij} = L_{ji} \leq 0, L\mathbf{1} = \mathbf{0}$$

Nous allons mettre ce problème sous forme quadratique pour pouvoir utiliser les outils d'optimisation classique de matlab.

tout d'abord, on va définir les notions suivante de la vécotorisation des matrices.

- Vectorisation et demi-vectorisation:

soit $M = (m_{ij})_{1 \leq i, j \leq n}$ une matrice symétrique alors la vectorisation de M est le vecteur colonne noté $vec(M)$ est définie par:

$$vec(M) = (m_{11}, m_{21}, \dots, m_{n1}, m_{12}, m_{22}, \dots, m_{n2}, \dots, m_{1n}, m_{2n}, \dots, m_{nn})^T \in \mathbb{R}^{n^2}.$$

La demi-vectorisation de M est le vecteur colonne noté $vech(M)$ et défini par:

$$vech(M) = (m_{11}, m_{21}, \dots, m_{n1}, m_{22}, m_{32}, \dots, m_{n2}, \dots, m_{ii}, m_{i+1,i}, \dots, m_{ni}, \dots, m_{n-1,n-1}, m_{nn})^T \in \mathbb{R}^{\frac{n(n+1)}{2}}.$$

- La matrice de duplication

La matrice de duplication d'ordre n noté $\mathfrak{M}_n$ est l'unique matrice de taille $n^2 \times \frac{n(n+1)}{2}$ qui à pour toute matrice symétrique de taille $n \times n$, $\mathfrak{M}_n vech(M) = vec(M)$.

D'après [3], on peut donner une formule explicite de $\mathfrak{M}_n$ par:

$$\mathfrak{M}_n = \sum_{i \geq j} u_{ij} vec(T_{ij})^T$$

avec $T_{ij} = E_{ij} + E_{ji}$ si $i \neq j$ et $T_{ii} = E_{ii}$, ou $(E_{ij})_{1 \leq i, j \leq n}$ est la base canonique de $M_{n \times n}(\mathbb{R})$. Et u_{ij} est un vecteur unitaire de taille $\frac{n(n+1)}{2}$ qui a la valeur 1 à la position $(j-1)n + i - \frac{j(j-1)}{2}$ et 0 partout (il faut pas oublier que u_{ij} est définie pour $i \geq j$).

Revenons maintenant au problème (1), on peut vérifier par le calcul que $\|L\|_F^2 = vech(L)^T \mathfrak{M}_n^T \mathfrak{M}_n vech(L)$ et que $Tr(PLY^T) = vech(Y Y^T)^T \mathfrak{M}_n vech(L)$.

On pose $H = \mathfrak{M}_n^T \mathfrak{M}_n$, $q = \mathfrak{M}_n^T vech(Y Y^T)$ et $z = vech(L)$. puisque à partir de z on peut obtenir L (car L est symétrique), alors le problème (1) est équivalent à:

$$\min_{z \in \mathbb{R}^{\frac{n(n+1)}{2}}} \alpha q^T z + \beta z^T H z \quad (2)$$

$$t.q \quad Az = 0, Bz \leq 0$$

avec A et B sont des matrices sparses qui expriment les conditions d'égalités et des inégalités respectivement dans le problème (1).

Après avoir résoudre ce problème, on peut passer à la deuxième partie qui consiste en la détermination de Y en fixant cette fois L via le problème d'optimisation suivant:

$$\min_{Y \in M_{n \times p}(\mathbb{R})} \|X - Y\|_F^2 + \alpha \text{Tr}(YLY^T) \quad (3)$$

D'après les littératures [4], ce problème admet une solution de la forme $Y = (I_n + \alpha L)^{-1} X^T$.

Enfin, voici un pseudo-code de l'algorithme d'apprentissage:

```

Input :  $X, \alpha, \beta$  et  $\maxIter$ 
output :  $L, Y$ 
step 1 : initialiser  $iter = 0, Y_{iter} = X$ 
while  $iter < \maxIter$ 
    résoudre (2) pour obtenir  $L_{iter}$ 
    résoudre (3) pour obtenir  $Y_{iter}$ 
end while
 $L = L_{iter}, Y = Y_{iter}$ 

```

6 Application à MNIST

6.1 Description

Nous allons appliquer maintenant ce que nous avons vu à la base de données MNIST, nous procéderons comme suit:

d'abord nous allons construire une matrice de descripteurs associée à chaque chiffre de 0...9, en suite on va appliquer l'algorithme vu ci-dessus pour obtenir une matrice Laplacian pour chaque chiffre, on obtient donc une famille de matrice Laplacian $\mathcal{L} = (L_0, \dots, L_9)$.

L'étape suivante c'est celle des tests, il faut prendre une image quelconque, calculer son descripteur et ensuite prédire la valeur de son label (chiffre), la méthode que nous avons utilisé pour la prédiction est: $label = \arg \min_{0 \leq i \leq 9} x^T L_i x$ avec x le descripteur de l'image test concernée.

6.2 Résultats

Lors de la programmation, on a utilisé deux bibliothèques matlab pour résoudre le problème d'optimisation de la partie 5.2, la première est quadprog est la deuxième est cvx_opt, la deuxième est très rapide et elle fournie des précisions plus bonnes, tandis que la première ne marche pas si la dimensions des descripteurs est plus de 90.

Avec la bibliothèque cvx_opt, on peut même tester avec des descripteurs de dimensions 768 (le programme dans ce cas prend un peu de temps), mais les résultats dépendent de cette dimensions et le nombre de signaux.

Les remarques qu'on a obtenu est que lorsque le nombre de signaux est très grand, la précision réduit, et lorsque la dimension est très grande (plus que 700), la précision également réduit.

Nous avons tester l'algorithme à d'autres bases de données telles que Yale et ORL et on a obtenu des précision de 1.

7 Conclusion

Les algorithmes de machine learning sont des outils très efficaces pour l'apprentissage, toutefois, ces algorithmes diffèrent par leurs résultats fournis, comme c'est le cas pour l'algorithme présenté dans ce qui précède qui donne une précision 0.91 pour l'apprentissage de la base MNIST, mais peut être que cet algorithme pourra être amélioré pour aboutir à des résultats plus significatifs (une précision de 1 par exemple pour la base MNIST).

References

- [1] Pascal Frossard Antonio Ortega et Pierre Vandergheynst David I Shuman, Sunil K. Narang. The emerging field of signal processing on graphs. 2013.
- [2] Navneet Dalal et Bill Triggs. Histograms of oriented gradients for human detection.
- [3] Jan R. Magnus et H. Neudecker. The elimination matriix: some lemmas and applications. 1980.
- [4] Pascal Frossard et Pierre Vandergheynst Xiaowen Dong, Dorina Thanou. Learning laplacian matrix in smooth graph signal representations. 2015.